

Hubs × Palantir — likheter, skillnader och vad vi kan låna

ITSL · Det operativa verksamhetslagret vs Palantirs plattformsmodell · Underlag: kodläsning (Nextcloud-vanilla, itsl-open-stack) + webbresearch (14 agenter)

1. Sammanfattning

Känslan stämmer — och den är djupare än den ser ut. Palantirs kärnaffär är inte "big data" utan exakt det Hubs bygger: ett **operativt lager mellan spretiga källsystem och operatörens beslut**, bestående av en begreppsmodell (objekt, relationer), en handlingsmodell (validerade åtgärder som skriver tillbaka till källsystemen) och en styrningsmodell (behörighet, spårbarhet, motiveringstvång). Hubs ärendemotor är, utan att kalla sig det, **en ontologi i miniatyr**: `Arende / Part / Member` är objekttyper, `Pekare` är länkar, `sagan / tilldelat / commit` är action types, och Treserva-committen med verifierad callback är writeback. Även affärssidan rimmar: `ArendeTyp`-tabellen är ITSL:s "gravel roads → paved highways", `itsl-CLI`:t ett embryonalt Apollo, Nate Open Stack ett embryonalt AIP, och demo-läget + mallbiblioteket råmaterialet till en bootcamp.

Men analogin har en avgörande spegelvändning, och den är Hubs viktigaste tillgång: **Palantir kopierar in kundens data och gör kopian till sin vallgrav; Hubs vägrar per doktrin** (NEVER-SoR, "kartan, inte territoriet", gallring efter verifierad commit). Palantirs ontologi vill vara en digital tvilling av verksamheten — det är precis den masskorreleringsförmågan som tyska författningsdomstolen underkände lagstödet för, som driver NHS-inlåsningsdebatten och som gör Palantir svårsåld i Europa. Hubs "ontologi" är en ontologi **över koordination och process, inte över verksamhetens fakta**.

Slutsatsen i en mening: **stjäl Palantirs begreppsapparat och discipliner — ontologi, actions, writeback, markings, checkpoints, audit, bootcamp-ritualen, produktifierad leverans — men inte deras datagravitationsmodell**. Sälj inte "svensk Palantir"; sälj den motsatta arkitekturen: samma operativa mekanik, spegelvänd vallgrav.

2. Vad Palantir faktiskt säljer — kort anatomi

Fyra produkter som paketeras som "operativsystem för organisationen":

Foundry + Ontologin. Kärnan är ontologin: ett operativt lager ovanpå integrerad data. Fyra begrepp: **object types** (substantiven: kund, order, patientsäng — schemadefinierade entiteter), **link types** (förstklassiga, navigerbara relationer), **action types** (verben: en validerad, transaktionsliknande uppsättning ändringar med sideeffekter — t.ex. "Assign Employee" ändrar rollattribut och skapar chefslänk i samma operation) och **functions** (affärslogik). Palantir delar uttryckligen ontologin i ett **semantiskt lager** (kunskapsmodellen) och ett **kinetiskt lager** (förändringsmodellen) — deras tes är att kunskapsmodellen är värdelös utan förändringsmodellen. **Writeback**: en action kan koppla en webhook mot källsystemet som körs *före* ontologiändringen — misslyckas anropet ändras inget internt, och källsystemets svar (t.ex. ett ID) skrivs in i objektet. (Obs: det är en enklare garanti än äkta tvåfas-commit, men mönstret är sunt.) Data landar via pipelines med **lineage** (spårbarhet råkälla → beslut) och indexerats kontinuerligt av tjänsten Funnel. Användaren möter ontologin i **Object Explorer** (sök, "search around" längs länkar), **Object Views** (navet för

ett objekt), **Workshop** (low-code-appar där operatören ser objekt och trycker på actions) och **Quiver** (analys/dashboards).

Gotham. Analytikerplattformen för polis/underrättelse/försvär: entitetsgraf (person, fordon, händelse, dokument) + karta + tidslinje, "link, map, filter, query", proveniens på varje påstående. Europeiska avtryck: NHS Federated Data Platform i UK (Foundry, £182M–£330M — nu under politisk omprövning med break-klausul 2027), hessenDATA i Tyskland (där författningsdomstolen 2023 underkände **lagstödet** för automatiserad dataanalys — inte tekniken i sig), danska POL-INTEL (~12 sammankopplade register, dokumenterad ändamålsglidning), och norska polisens avbrutna projekt (~100 MNOK, ingen leverans).

AIP. LLM/agent-lagret ovanpå ontologin (2023–). Nyckelmönstren: **objekt som kontext** i stället för text-RAG ("Ontology-Aware Generation"), **actions som verktyg** (agenten kan aldrig skriva fritt — alla skrivningar går genom fördefinierade actions med validering och valbar användarbekräftelse), **ärvda behörigheter vid inferenspunkten** (agenten ser bara vad den anropande användaren får se), **AIP Evals** som produktionsgrind (testfall körda ≥ 3 gånger p.g.a. icke-determinism), och human-in-the-loop för höginsatsbeslut. Mycket av detta är Palantirs egna claims med tunn oberoende verifiering — men designspråket är rätt.

Apollo. Leveransplattformen: autonoma zero-downtime-uppgraderingar över moln, on-prem och air-gappade miljöer. Palantir krediterar uttryckligen Apollo för att marginalerna blev SaaS-lik (80 % brutto) i stället för konsultlika — deployment, inte koden, var det som gjorde dem "konsultiga".

Leveransmodellen. Forward Deployed Engineers bäddas in hos kunden och bygger overfittade lösningar; ett separat produktteam generaliserar mönstren till plattformsfunktioner ("gravel roads → paved highways" — så uppstod hela Foundry). **AIP Bootcamps** (1–5 dagar, kundens egen data, fungerande workflow i stället för slides) komprimerade säljcykeln från ~12 månader till dagar; 1 300+ genomförda per Q4 2024 med ~75 % konvertering enligt *Palantir själva*. **Acquire → Expand → Scale**: gratis/billiga piloter med negativ marginal, expansion per use case, mjukvaruekonomi först i Scale-fasen — en modell som kräver kapital att bränna och som i offentlig sektor upprepade gånger kritiserats som otillåten förhandsbindning.

Styrningsprimitiver (det mest lånbara): **markings** (obligatoriska skyddsetiketter som *propagerar med härledd data* och inte kan delegeras bort), **purpose-based access control** (åtkomst knuten till deklarerat ändamål, med motiveringsplikt åt båda håll), **restricted views** (radnivåssäkerhet), **append-only audit** (varje interaktion, strömlinje till extern SIEM så tillsynen kan ligga utanför plattformen), **data lineage** och **Checkpoints** — användaren tvingas ange motivering i fritext *innan* en känslig åtgärd genomförs (60+ interaktionstyper: sökning, export, delning), och motiveringarna granskas periodiskt.

3. Vad Hubs är idag — spegeln

Hubs tes: kommunen har redan sin juridiska slutlagring (facksystemet — Treserva, Lifecare, Viva, Combine ...), men *vägen dit* är trasig. Det operativa verksamhetslagret är den säkra arbetsytan däremellan: en åtgärd-först-vy (Min dag), en ärendemotor, ärenderum, frister, kvittenser, gallring — allt i kommunens egen drift.

Motorns informationsmodell (`hubs_arende`): **Arende** (koordinationsraden med identitetstrippeln `conversationId / hubsCaseId / dnr` — tre id:n med tre olika ägare), **Pekare** (tvåvägslänkar till Deck-kort, Talk-rum, groupfolder, kalenderobjekt, team — opaka referenser, aldrig innehåll), **Member** (förstklassigt medlemskap med roller), **ArendeTyp** (datadriven typkonfiguration: ny ärendetyp = ny rad, inte ny PHP — verifierad mot 12 kommunala roller), **Handelse** (PII-fri händelsejournal), **Part** (motorns enda sanktionerade PII-tabell, gallras med ärendet, fail-closed skyddsnivåer) och **Sakuppgift** (bekräftade fakta för förfyllnad). Sagan i `ArendeService::createCase` orkestrerar tio steg med kompenserande closures; `commit()` skriver till facksystemet via port med **verifierad callback som enda utlösare av retention** (GAP-007);

`GallringService` purgar motorns egen rad först efter verifierad commit. Doktrinen är hård och kodhävdat: **Hubs är aldrig System of Record** — `commitDestination NOT NULL` på varje ärendetyp är den strukturella garantin mot skuggregister.

Frontend: Min dag är åtgärd-först (dagspuls, triageband, varsellista, frist-sorterade ärendekort), och `NastaAtgardKnapp` härleder EN ledande åtgärd per steg ur en state machine i `arendeFlow.js` — med pliktgrindar (skyddsbedömning) och CommitGrind/GallringsGrind som mänskliga kontrollpunkter.

Bredvid detta: **Nate Open Stack** (itsl-open-stack) — fyra personbundna agenter med egna brains (pgvector), ingestionsmotor över sex källor till ett Evidence-schema med PII-brandvägg, skills/runbooks med aktiveringsmodell (ägargodkännande, BankID-re-auth vid scope-expansion) — idag ett internt verktyg för ITSL:s egen organisation, medvetet inte kundvänt än.

Kravläget (nattanalysen 2026-07-05): motorn och socialsekreterar-UI:t är starka; svagast är K-5 (noll live-konnektorer) och K-7 (åtkomstlogg, PuB-matris, signeringsbevis — "juridiskt osäljbar" utan), plus P0-fynd om inre-sekretess-läckor på läsvägen och en kedja där stub-fallback kan fabricera verifierad commit → olaglig gallring.

4. Likheterna — punkt för punkt

Palantir	Hubs	Kommentar
Object types (substantiven)	<code>Arende</code> , <code>Part</code> , <code>Member</code> , <code>Handelse</code> , <code>Sakuppgift</code>	Schemadefinierade entiteter med tydligt ägarskap
Link types ("search around" utan join-fan-out)	<code>Pekare</code> + <code>case:{id}</code> -tagg åt andra hållet	Samma funktion: "öppna ärendet" är O(1), inte fan-out över sju appar
Action types (validerade verb med sideeffekter)	Sagan, <code>tilldela()</code> , <code>commit()</code> , lifecycle-transitioner med grindar	Hubs actions finns — men som PHP-metoder, inte som deklarerat register (se §6.3)
Writeback (webhook före intern ändring; källsystemets svar in i objektet)	<code>FacksystemCommitService</code> → port → verifierat kvitto → <code>dnr</code> in i registret	Samma semantik; Hubs binder dessutom retention till kvittot
Semantic + kinetic layer	Registret + NastaAtgard-state-machinen	Hubs kinetiska kunskap bor delvis i frontend i dag
Interfaces/polymorfism	<code>ArendeTyp</code> som parametriserar en generisk saga	"Verksamhetsskillnad = modellering, inte kod" — samma designfilosofi
Workshop / Object Views / Object Explorer	Min dag / ärendekortet med flikar / summary-aggregatet	Hubs får applikationslagret "gratis" av Nextcloud-apparna
Checkpoints, human-in-the-loop	CommitGrind, GallringsGrind, pliktgrind	Hubs har mönstret på skrivvägen — saknar det på läsvägen (se §6.1)
FDE → "gravel roads → paved highways"	Socialsekreterare-vertikalen → <code>ArendeTyp</code> -vokabulären (12-rollersverifieringen gav ~7 nya config-fält)	Exakt samma destillationsloop, i mikroformat
Apollo	<code>itsl</code> CLI (start/stop/update/deploy)	Rätt frö, fel mognad — se §7.2
AIP (objekt som kontext, actions som verktyg, ärvda permissions, evals)	Nate Open Stack (brains, skills, runbooks, PII-brandvägg, Citation Guard, tvåtiers-säkerhet)	Strukturellt samma designspråk; Nate är internt, inte kundvänt — klok sekvensering

Palantir	Hubs	Kommentar
Bootcamp (1–5 dagar, fungerande workflow som säljverktyg)	DemoSeedService + ?demo=1 + mallbibliotek + Collectives-handbok	Alla artefakter finns; ritualen och paketeringen saknas
"OS för organisationen", en plattform × N domäner	En motor × 12 kommunala roller på samma driftmiljö	Samma horisontella tes: bredd utan vertikala punktlösningar

Det är alltså inte en ytlig likhet. Båda initiativen har oberoende landat i samma treenighet — **begreppsmodell + handlingsmodell + styrningsmodell ovanpå källsystem som behålls** — därför att det är den form som operativt beslutsfattande i informationstunga organisationer faktiskt kräver. Palantir är beviset i stor skala för att den formen bär mjukvaruekonomi även i tung offentlig sektor.

5. Den avgörande skillnaden: sanningsriktningen

Palantir kopierar in; Hubs pekar ut. Foundrys pipelines synkar källdata in i plattformen och indexerar den till objektdata (‐tens of billions of objects‐); ontologin blir platsen där verksamheten ser och beslutar, och källsystemen degraderas till foder och exekveringsändrar. Det är designat: analytiker läser tvillingmetaforen som Palantirs vallgrav — ontologin ackumulerar kundens affärslogik och gör exit till en total ombyggnad (NHS-kontraktets §16.1 ger inte NHS IP till specialbyggd kod; NYPD fick vid sin exit strid om att få ut sitt eget analysdata).

Hubs valde spegelbilden, av juridiska skäl som visat sig vara strategiska: motorn håller pseudonymt koordinationsstate, innehållet bor i facksystemet respektive i Nextcloud-appar som motorn bara pekar på, och Hubs-kopian *gallras* efter verifierad commit. Där Palantirs objekt är innehållsbärande kopior är Hubs pekare opaka referenser. Konsekvensen: **där Palantir måste bevisa att plattformen inte används för masskorrelering, kan Hubs visa att motorn inte förmår det.** Inför IMY, en DPIA-granskare eller en förvaltningsdomstol är det en kvalitativt starkare position.

Två ärlighetsnyanser som måste med för att argumentet ska hålla:

- 1. ‐Kartan, inte territoriet‐ gäller motorn — inte plattformen.** Nextcloud-instansen som helhet innehåller territoriet: filer i ärenderum, Talk-trådar, Deck-kort och kalendrar med verksamhetsinnehåll och PII. Nextclouds fulltextsök/unified search över ärenderum är en faktisk korreleringsyta, och en admin med serveråtkomst kan korsläsa. Skillnaden mot Palantir är fortfarande verklig (ingen entitetsupplösning, ingen tvärgraf, ACL:er per ärenderum, allt i kommunens egen drift) — men DPIA:n måste beskriva plattformen, inte bara motorn, och sök-/adminytorna måste styras av samma sekretessgränser som allt annat.
- 2. Den spegelvända vallgraven är också en ekonomisk risk.** Palantirs stickiness kommer av datagravitation; Hubs avsäger sig den medvetet. ITSL:s kvarhållning måste komma från arbetsflödesvärde, förtroende, leveransexcellens och konnektorunderhåll — svagare naturliga krafter. Det ska sägas ärligt internt, och kompenseras aktivt (se §7).

Därtill skalskillnaden: Palantirs styrningsprimitiver förvaltas av dedikerade governance-team hos kunder med hundratals analytiker. Hubs måste designa för att granskningen görs av en deltids-DSO i en mellanstor kommun — aggregatvyer och undantagslistor, inte loggfloder. Låna mönstret, inte maskineriet.

6. Vad Hubs bör låna — tekniskt, rangordnat

Rangordnat efter juridisk hävstång × genomförbarhet för ett litet bolag. Punkt 1–3 adresserar direkt nattanalysens allvarligaste fynd.

6.1 Beständig åtkomstlogg + Checkpoints på läsvägen ★ störst hävstång

Palantirs Audit.3 + Checkpoints är den tekniska formen av socialtjänstens klassiska regel "slå bara i registret med dokumenterat skäl". Hubs har grindar på *skrivvägen* (CommitGrind, GallringsGrind, pliktgrind) men K-7-gapet på *läsvägen*: åtkomstloggen är 1/36 täckt, och `Handelse`-journalen gallras med ärendet — vilket är precis fel för ett auditspår: **en logg som raderas när ärendet gallras kan aldrig bevisa att gallringen var laglig.**

Design: separera **verksamhetsjournalen** (`Handelse` — gallras med ärendet, GDPR-korrekt) från en ny **åtkomst-/åtgärdslogg** (pseudonym, append-only, överlever gallringen, exporterbar till kommunens egen logggranskning). Ovanpå den: generalisera grind-mönstret till en `Grind`-komponent + `hubs_arende_checkpoint`-tabell med triggerpunkter i prioritetsordning: (1) åtkomst till ärende utanför egen medlemskrets — "bryta glaset"-flöde med deklarerat skäl, tidsbegränsat fönster och notis till handläggaren; (2) läsning av `Part` med `skydd ≠ ingen`; (3) export/nedladdning ur ärenderum; (4) sökning som träffar över `enhet`-gränsen; (5) medlemstillägg/omfördelning (stänger samtidigt laggTillMedlem-self-add-fyndet). Plus en månatlig aggregatvy för dataskyddsombudet, byggd på `StatusService`-mönstret.

Detta är dessutom säljbart *mot* Palantir: "deras audit-disciplin, utan deras datainsamling — och er DSO kan läsa koden som upprätthåller den." Obs (arkivjuridik): loggen är själv en allmän handling med egen gallringsfrist — den behöver en dokumenterad informationshanteringsplan, inte "för evigt".

6.2 Gallringskvitton + verifierad radering av alla projektioner

Nattanalysen fann att gallringen "river kartan, ej datat" — pekarna raderas men externa objekt kan överleva. Palantir-lärdomen är lineage-tänket: Hubs `Pekare`-tabell *är* redan en lineage-graf (sagan vet exakt vilka objekt som härletts ur ärendet). Låt gallringen producera ett **kvitto per pekare** (objekt X raderat / kunde ej raderas, tidsstämplat) i den beständiga loggen, och fail-closed på integrationsläge så att stub-fallback aldrig kan returnera `verifierad=true` i produktion. Då blir kedjan sluten: verifierad commit → verifierad gallring → beständigt kvitto.

6.3 Action-register i `ArendeTyp` — gör det kinetiska lagret explicit

Kunskapen om "vilka verb är lagliga i vilket steg" finns redan — men i Vue-koden (`arendeFlow.js` / `NASTA_ATGARD`). Palantir-lärdomen: deklarera actions per ärendetyp och steg i motorn (namn, behörighetskrav, grind, sidoeffekter — JSON-config i typregistret) och låt frontend bli en projektion av det. Tre vinster: GUI:t kan aldrig visa en åtgärd servern inte tillåter; registret blir enumererbart för tillsyn ("visa alla åtgärder rollen X kan utföra"); och det är förutsättningen för att någonsin exponera actions säkert för agenter (§6.6).

6.4 Typade länkar + fler objekttyper som koordinationsskuggor

`Pekare.riktning` är i dag överlagrad (betyder boardId för Deck, ägar-uid för kalender). Ersätt med `relationTyp` + `relationMeta` — liten migrering, stor begreppslig vinst. Därefter: **Beslut** och **Insats** som egna entiteter — men som *koordinationsskuggor* (lagrum, datum, beslutsfattare-referens, frist, pekare till handlingen i facksystemet — aldrig innehållet). Det ger uppföljningszonen och verkställighetsbevakning riktig substans ("ej verkställda beslut" är dessutom en IVO-rapporteringsplikt), fortfarande NEVER-SoR-rent. Relationer mellan parter (vårdnadshavare–barn, hushåll, syskonärenden) hör hit — som typade länkar, inte som fritext.

6.5 Sekretessmarking som propagerar via sagan

Palantirs viktigaste styrningsidé: skyddsklassningen *reser med* data — skapas ett derivat följer markingen med, obligatoriskt. Hubs skapar sju kända projektioner per ärende via en enda saga — låt sagan stämpla en sekretessmarking (härledd ur `ArendeTyp.sekretessGrund` + `Part.skydd`) på varje skapat objekt, och låt läsvägen verifiera den. Det gör OSL-klassningen till en egenskap hos *objektet*, inte hos lagringsplatsen, och är grunden för sekretessmurar (SoL ↔ HSL, EMI ↔ skola). Kräver disciplin i alla integrationsklienter, men `Pekare` - infrastrukturen finns redan.

6.6 Ontologin som agentyta (AIP-mönstret) — i rätt sekvens

AIP:s mönster mappar punkt för punkt mot det ni redan byggt: objekt som kontext (summary-aggregatet), actions som verktyg (registret från 6.3), ärvda behörigheter (NC-behörighetsmodellen), evals-grind (Citation Guard är embryot), human-in-the-loop (grindarna). Det som saknas är kontraktet mellan Hubs-motorn och Open Stack-agenterna: exponera **läsning** (händelsejournal, pekare, frister — "sammanfatta ärendeläget") som MCP-verktyg med ärvda behörigheter först; **skrivning** endast genom action-registret med obligatorisk användarbekräftelse, aldrig utan.

Rättslig ram som måste med innan något kundvänt agentsteg: EU:s AI-förordning klassar system som styr tillgång till väsentliga offentliga tjänster som högrisk (bilaga III), GDPR art. 22 och förvaltningsrätten begränsar automatiserat beslutsfattande, och Trelleborg-fallet (RPA-beslut i ekonomiskt bistånd) är den svenska varningsberättelsen. En läsagent är okontroversiell; allt som närmar sig bedömning eller beslut kräver egen rättslig analys. AIP-sekvensen — ontologi först, agenter sedan, läsning före skrivning — är juridiskt tvingande här, inte bara god ingenjörssed.

6.7 Person-nyckel över ärenden — det farligaste och mest värdefulla steget

"Samma barn, tredje anmälan på två år" är kanske det enskilt största barnskyddsvärdet som dagens modell inte kan leverera — `Part` är per-ärende och gallras med det, så återkommande personer är osynliga för motorn. Det är också **exakt här Palantir-arkitekturen börjar**: en persistent person-identitet som länkar över ärenden är fröet till registret som BVerfG- domen handlade om. Om det byggs: pseudonym nyckel, aldrig fritt sökbar, endast som grindad funktion bakom motiveringstvänet i 6.1, med rättslig analys (OSL inre sekretess, ändamålsprövning) *före* design — tröskeln måste finnas i regelverket, inte bara i tekniken. Bygg 6.1 först; utan åtkomstlogg är denna funktion oförsvarbar.

6.8 Reconciliation-sweep (Funnel-light)

Palantirs Funnel håller ontologin kontinuerligt i synk; Hubs projektioner synkas best-effort och kan divergera. En periodisk sweep som jämför pekare mot faktiskt NC-state och loggar drift är Funnel-idén i liten skala — lågt hängande driftskvalitet.

7. Vad ITSL bör låna — affär och leverans

7.1 Paketera "Hubs-dagen" (bootcamp-ritualen)

Palantirs bootcamp-insikt: POC:en är säljverktyget — kunden ska *se sin egen verklighet fungera*, inte slides. Alla artefakter finns: `DemoSeedService` kör tio syntetiska ärenden genom den riktiga motorn, `?demo=1` ger fixtures utan serverändring, mallbiblioteket + Collectives-handboken är leverabeln kunden behåller. Formatet: förmiddag — Min dag i kommunens egen testmiljö; eftermiddag — konfigurera *kommunens egen ärendetyp* som `ArendeTyp` -rad live, ladda deras mallar, visa kedjan inflöde → ärenderum → mall → commit → kvitto. Den

datadrivna typkonfigurationen är ett dolt bootcamp-vapen: "er ärendetyp på tio minuter" är en demonstration Palantir behöver en FDE-vecka för.

Två skillnader mot förebilden är tvingande: **syntetisk data** (aldrig kundens riktiga — omöjligt i socialtjänsten, och rätt även säljmässigt) och **LOU-medvetenhet** — gratis-pilot-mönstret är i offentlig sektor en riskvektor (UK:s upphandlingschef kritiserade Palantirs nollprispraxis; New Orleans körde predictive policing i sex år som "gåva" utan fullmäktiges kännedom). Hubs-dagen ska designas som *marknadsdialog/demonstration*, inte som förhandsbindande leverans, och den öppna kärnan är det legala svaret: kommunen kan granska och testa utan att köpa något.

7.2 Lyft `itsl` CLI mot "Apollo-light"

Palantirs viktigaste ekonomiska lärdom: det var inte koden som gjorde dem konsultiga utan *leveransen*, och Apollo är det som gav mjukvarumarginaler i on-prem-miljöer. Kommunal drift är strukturellt samma problem som Palantirs klassificerade miljöer: varje kund kör egen instans i egen driftmiljö. Gapet mellan "CLI som Fredrik kör mot dev15" och "flotta av 30 kommuninstanser som uppdateras på en eftermiddag av en driftpartner, med hälsokontroll, rollback och deploy-kvitton" är den enskilt viktigaste tekniska investeringen för skalbar ekonomi — och "förvaltningsbarhet" är dessutom ett säljbart löfte i upphandlingar i sig.

7.3 Formalisera FDE-loopen i mikroformat

Loopen finns redan informellt (socialsekreterare-vertikalen → 12-rollersverifieringen → ~7 nya config-fält). Det som saknas är disciplinen: varje kundspecifik lösning prövas inom en sprint mot frågan "ny `ArendeType` -parameter eller engångsfix?" — så att kunddomänens tysta kunskap matar config-vokabulären, inte en fork. En namngiven verksamhetsnära ingenjör (deltid räcker) per referenskommun.

7.4 Sälj reversibilitet som produktgenskap — vapnet Palantir inte kan kopiera

Gör anti-inlåsning till artefakter, inte adjektiv: publicerat exit-protokoll med **testad exit som leveransmoment år 1**, kundägd config (`ArendeType`-rader + mallar exporterbara i maskinläsbart format), AGPL-kärna, offentlig DPIA-mall, och arkitekturprincipen "er data har aldrig varit hos oss". Varje NHS-rubrik och varje BVerfG-referens är gratis marknadsföring för den positionen — och den harmonierar med suveränitetsvågen (openDesk, med Nextcloud som komponent, väljs redan av tyska delstater). Men ta netzpolitik-varningen på allvar: en europeisk klon av korreleringsarkitekturen ärver Palantir-problemen. Pitchen är "vi byggde aldrig det domstolen fällde", inte "Palantir fast svenskt".

7.5 "AIPCon för kommuner" i miniatyr + delbara artefakter

Palantirs expansionsmotor efter bootcampen är AIPCon: kunder demoar för varandra. Kommunsektorn köper på peer-bevis mer än leverantörsdemos — 290 kommuner som pratar med varandra är en liten och skvallrig marknad (risk och tillgång på samma gång). Ett årligt användarnätverk där kommuner visar sina egna ärendetyper och mallar, plus **delbara `ArendeType`-definitioner och mallpaket som ekosystemartefakter** (Hubs motsvarighet till Palantirs Marketplace), kostar en dag och en lokal. Kommunalförbund och gemensamma driftorganisationer är dessutom en go-to-market-kanal som ger flera kommuner per affär.

7.6 Sekvensen — Palantir-läxan i negativ

Palantir bootcampade en färdig plattform; Hubs är inte där. En bootcamp som slutar i "konnektorn är en stub" konverterar inte — den bränner referensmarknaden. Därför är ordningen krass: **K-7-lagret (§6.1–6.2) och EN live-konnektor i EN referenskommun före all expansionsteater**. En lysande referenskommun slår tio prospekt. Och NHS-läxan därtill: bootcamp-konvertering ≠ användning — FDP:s adoptionskris (läkarbojkott, trusts som tappat funktionalitet) visar att professionens acceptans är den verkliga grinden. Hubs åtgärd-först-

design med "värde på första klicket" är rätt svar, men den ska mätas i verklig användning hos referenskommunen, inte i sålda licenser.

7.7 Nate-stacken: mogna internt, externalisera medvetet

Att Nate Open Stack körs på ITSL:s egen verksamhet först är klokare sekvensering än de flesta AIP-kopior — ITSL kör i praktiken sitt eget interna bootcamp på agentmodellen. Tidsätt externaliseringen: när aktiveringsmodellen, PII-brandväggen och runbook-exekveringen är bevisade i drift definieras det första kundvända steget smalt (läsagent per §6.6), aldrig tidigare.

8. Varningskatalogen — Palantirs misslyckanden som designregler

Händelse	Läxa för Hubs
BVerfG 1 BvR 1547/19 (feb 2023): Hessens/Hamburgs lagstöd för automatiserad dataanalys författningsstridigt — för låga ingreppströsklar, "praktiskt taget inga begränsningar av typ och mängd data", profiler "med ett klick". Domen fällde lagstödet, inte tekniken	Tekniska kontroller räddar inte en användning utan rättslig tröskel. Behandla <i>frånvaron</i> av tvåanalys som en dokumenterad feature (DPIA, arkitekturbeslut). Partsregistrets rollseparation (barn/vårdnadshavare/anmälare/motpart) är juridiskt bärande — platta aldrig till den
NHS FDP : inläsningsklausuler (§16.1), maskade kontrakt, break-klausul övervägs 2027, läkarbojkott, <hälften av trusts i verklig användning	Kundägd datamodell + testad exit + professionens acceptans som mätetal. Transparens som default: publika kontrakt, öppen kod, offentlig DPIA
NYPD-exit : vägran att exportera analysresultat i standardformat	Exportbarhet är en produktegenskap, inte en eftergift — bygg och <i>demonstrera</i> den
POL-INTEL : "supervapen mot terror" gled till vardagsbrott; lagar ändrades för att mata systemet	Function creep är default, inte undantag. Ändamålsbegränsning ska vara datamodell (<code>ArendeTyp</code> , <code>commitDestination</code>), inte policy
New Orleans : sex år hemlig "pro bono"-drift utan fullmäktige	Ingen pilot utan upphandlingsmässig och demokratisk förankring
Norge : ~100 MNOK, avbrutet utan leverans	Integrationslöften är projektets risk — därav "EN live-konnektor först"
Homes for Ukraine / Met Police : gratis → £10M+; £15–25M-estimat → £50M blockerat	Nollpris-ingång skapar kostnadsexplosion senare; transparent modulprissättning (M0–M4 + konnektorfamiljer) är motmedlet
LAPD LASER : nedlagt — data "validerade befintliga polismönster"	Varje framtida analys-/prioriteringsfunktion behöver bias-granskning; socialtjänstdata bär samma risk
Den tysta signalen : Palantir har efter 20+ år nästan ingen närvaro i kommunal socialtjänst	Delvis för att affären är för liten för deras kostnadsstruktur — vilket är ITSL:s lucka, men också en varning om marknadens betalningsvilja och upphandlingsfriktion. ITSL:s kostnadsstruktur måste förbli en bråkdel av Palantirs

9. Var Hubs står strukturellt starkare

1. **Datasuveränitet på riktigt.** Självhostad AGPL-stack i kommunens drift träffas inte av CLOUD Act-argumentet som även "Gotham Europa" och sovereign-cloud-konstruktioner brottas med. Noll tredjelandsöverföringar är ett upphandlingsargument, inte en teknisk detalj.

2. **NEVER-SoR minimerar attackytan per konstruktion** — med §5-nyansen ärligt redovisad (motorn vs plattformen).
3. **Granskningsbar styrning.** Kommunens DSO kan läsa exakt hur ACL:er sätts. NHS-kritiken "vi kan inte läsa koden som formar produkterna" är omöjlig här. Exploatera aktivt i upphandling.
4. **Ändamålsbegränsning som datamodell.** Palantirs kontroller är konfigurerbara av kunden — EFF:s kärnkritik är att konsolideringen upphäver ändamålsseparationen när huvudmannen vill. Hubs gör ändamålet till registrerad med tvingande invarianten; glidning kräver kodändring, inte en adminbock.
5. **Juridiken som schemafält.** Varje objekttyp och action kan bära lagrumsreferens och gallringsregel i schemat — det har Palantir inte; deras purpose limitation är konfiguration, er kan vara datamodell. (Detta är kanske den enskilt mest särskiljande produktiden i hela jämförelsen.)
6. **Rätt sida av historien.** DGSi har utvärderat franska alternativ (Athea/Argonos — uppgifterna om faktiskt byte är omstridda), BfV valde ChapsVision, Bundeswehr avstod Palantir, NHS-kontraktet omprövas. Suveränitetsvägen är verklig efterfrågan, och openDesk-spåret visar att Nextcloud-basen ligger rätt i den.

10. Bredare spår som jämförelsen öppnar

a) **Uppföljning och statistik — den största outnyttjade parallellen.** Palantirs starkaste offentliga säljargument är aggregerad verksamhetsstyrning. Hubs sitter på något facksystemen är svaga på: **processdata** (ledtider, fristefterlevnad, triagetider, volymer per ärendetyp, verkställighetstider) — pseudonymt, aggregerbart, IVO/Kolada/Socialstyrelse-relevant, och **nya socialtjänstlagen (2025:400) kräver kunskapsbaserad socialtjänst och systematisk uppföljning** — den enskilt starkaste svenska efterfrågedrivaren för exakt detta. StatusService-mönstret (endast aggregat, aldrig individdata) är rätt frö: ett "uppföljningslager" som producerar nämnd- och tillsynsunderlag ur koordinationsstatet vore Quiver-analogins kommunala form, utan att bli register. Kräver design så att aggregat inte blir bakvägsidentifierande i små kommuner (småtalsproblemet).

b) **Kommunontologin ska vara Socialstyrelsens — exekverbar.** Bygg inte en egen konkurrerande begreppsstandard: BBIC:s informationsspecifikation, IBIC, KSI och nationella informationsstrukturen finns redan. Säljargumentet är "vår ontologi ÄR Socialstyrelsens informationsspecifikation, körbar" — objekttyper och actions mappade mot nationella begrepp. Det ger också interoperabilitet gratis och avväpnar "proprietär modell"-kritiken innan den uppstår.

c) **Medborgarens vy av kartan.** Hela Palantir-kritiken drivs av de registrerades perspektiv — men det perspektivet kan Hubs *bygga för*: partsinsyn (den enskildes rätt att se sin akt), registerutdrag ur motorn + projektionerna, barnets rätt till information och delaktighet (barnkonventionen + nya SoL), status på "mitt ärende" via e-tjänst. En medborgarvy är en produktidé Palantir strukturellt inte kan erbjuda — deras slutanvändare är staten, aldrig den registrerade. För Hubs är det en förtroendeskapande differentiering och delvis lagkrav.

d) **Federation mellan kommuner.** Verkliga flöden korsar kommungränser: placerade barn i annan kommun, ärendeflytt, SIP-samverkan med regionen. Palantirs svar är multi-tenant-plattform; Hubs svar bör vara **federation** — Nextcloud har federationsprimitiver inbyggda, och SDK (Säker digital kommunikation via Digg) är den nationella kanalen som Hubs redan är byggt att tala med. 290 separata instanser utan federationsberättelse är annars en långsiktig svaghet; kommunalförbund/gemensam drift är bryggan.

e) **Ontologi som produkt över förvaltningar — testet på plattformstenen.** Palantirs kärnbevis är att samma ontologimotor bär flygbolag och sjukhus. Hubs motsvarande bevis är 12-rollersverifieringen: om **ArendeTyp**-modellen bär överförmyndarens årshjul och elevhälsans sekretessregim med config-rader är Hubs

en plattform, inte en socialtjänstapp. Horisontalexpansionen är alltså inte bara tillväxt — den är beviset för hela arkitekturtesen, och varje ny förvaltning som går in som "rad, inte fork" stärker det.

f) Ekosystem och nätverkseffekter. Palantirs vallgrav inkluderar Marketplace och utvecklar-SDK. Hubs spegelbild: konnektorer som tredjepartsbyggbara artefakter (kontraktet är redan portbaserat), delbara ärendetypsdefinitioner och mallpaket mellan kommuner, openDesk-spåret som europeisk kanal. Och spegelriskerna: ITSL:s beroende av Nextclouds roadmap (breaking changes i Deck/Talk-API:er) förtjänar en egen bevaknings- och abstraktionsstrategi — porterna/klienterna är redan rätt mönster, håll dem tunna och versionsmedvetna.

g) Konkurrensbilden, nyktert. Hubs verkliga konkurrent är inte Palantir — det är facksystemleverantörernas egna moderniseringssår (Treserva/Lifecare/Combine-portaler), "gör inget"-alternativet, och möjligen de europeiska Palantir-utmanarna (ChapsVision, Athea) om de rör sig nedåt mot kommunmarknaden. Positioneringen "stänger gapet inkorg↔akt, ersätter inte" är rätt just för att den inte tvingar kommunen välja bort sin facksystemsinvestering — behåll den disciplinerat, även när det frestar att bygga "bara en liten journalfunktion till".

11. Rekommenderad sekvens

Nu (veckor–månad):

1. Beständig åtkomstlogg (separerad från `Handelse`) + Checkpoints/bryta-glasat (§6.1) — stänger K-7-kärnan och är förutsättningen för allt annat.
2. Gallringskvitton + fail-closed integrationsläge (§6.2) — stänger nattanalysens allvarligaste kedja.
3. DPIA-/arkitekturdokumentation av "motorn vs plattformen"-nyansen (§5) inkl. unified search-ytan.

Kvartal: 4. Action-register i `ArendeTyp` + typade pekare (§6.3–6.4 första halvan). 5. `itsl` CLI → Apollo-light-målbild (§7.2) parallellt med EN live-konnektor i EN referenskommun (§7.6). 6. "Hubs-dagen" paketerad och LOU-granskad (§7.1) — körs först när 5 är sann.

Halvår+: 7. Sekretessmarkings via sagan (§6.5); Beslut/Insats som koordinationsskuggor + uppföljningslagret mot nya SoL (§6.4, §10a). 8. Agentkontraktet Hubs↔Open Stack: läsagent med ärvda behörigheter, AI Act-analys före allt kundvänt (§6.6, §7.7). 9. Person-nyckel över ärenden — endast efter 1, bakom motiveringstvång, med rättslig analys först (§6.7). 10. Ekosystemspåren: delbara typdefinitioner/mallpaket, kommunnätverk, federationsberättelse (§7.5, §10d, §10f).

Källor (urval)

Palantir primärt: palantir.com/docs — Ontology overview/core-concepts, Action types + webhooks, Object backend (Funnel/OSv2), Data Lineage, Security (markings, restricted views, audit-logs), Checkpoints, AIP/Logic/Agent Studio/Evals, Apollo; blog.palantir.com — "Connecting AI to Decisions", "Purpose-based Access Controls", "Palantir Apollo: powering SaaS where no SaaS has gone before", "A Day in the Life of a Forward Deployed Software Engineer"; SEC S-1 (Acquire/Expand/Scale). **Rättsligt/kritiskt:** BVerfG 1 BvR 1547/19 (dom + pressmeddelande, EN), Amnesty (2020, 2025), EFF (2026, ELITE/Medicaid), ACLU, Privacy International "All roads lead to Palantir", Medact FDP-briefing, netzpolitik.org om "europeisk Palantir", Brennan Center (NYPD), Type Investigations (New Orleans), The Intercept (LAPD LASER). **NHS/Europa:** Digital Health & The Register (break-klausul, BMA), NHS England FDP uptake, heise (BfV/ChapsVision, Bundeswehr), Lighthouse Reports, forskning om POL-INTEL (Tandfonline, Sage) och Norge (Tandfonline). **FDE/affär:** Nabeel

Qureshi "Reflections on Palantir", Pragmatic Engineer om FDE-modellen, GTM Foundry om bootcamp-strategin. **Hubs** **internt:** [analysis-output/VARDE-OPERATIVA-VERKSAMHETSLAGRET.md](#), [analysis-output/NATTANALYS-KRAV-2026-07-05.md](#), [analysis-output/KOMPLETT-ANALYS-2026-07-02.md](#), [hubs_start/docs/](#) (ARKITEKTUR, KRAVSTALLNING-TOTAL, KOMMUNROLLER-SOR, MODULARISERING-LICENS-DATALAGER, HUBS-INTERNALS-ARENDEMOTOR), [hubs_arende/lib/](#), [itsl-open-stack/docs/](#) (MASTERPLAN, KRAVSTALLNING-SKILLS-RUNBOOKS-GUI, KARTLAGGNING).

Siffror som ~75 % bootcamp-konvertering, "fler FDE:er än produktioningenjörer" och kundcase-ROI är Palantirs egna eller anekdotiska uppgifter och ska citeras som sådana.